

Plan de Análisis — Clasificación de Datos BCI

1. Objetivo del Plan de Análisis

El presente plan de análisis tiene como objetivo definir la metodología sistemática que permitirá evidenciar las diferencias en los ritmos cerebrales asociados a cinco condiciones experimentales: reposo (T0), imaginación de movimiento de mano derecha (T2), imaginación de movimiento de mano izquierda (T1), movimiento real de mano derecha y movimiento real de mano izquierda. Para ello, se combinarán las características espectrales obtenidas en el Proyecto 1 con los índices adicionales propuestos (CSP, entropía espectral y parámetros de Hjorth), y se construirán algoritmos de clasificación supervisados [1]–[3].

2. Descripción del Dataset y Condiciones de Interés

Se utilizarán los datos del dataset público Physionet EEG Motor Movement/Imagery Dataset (BCI2000), que contiene registros EEG de 109 sujetos bajo protocolos de imaginación y ejecución de movimiento [4]. Los registros de interés son:

- Runs R04, R08, R12 (Tarea 2): Imaginación de apertura/cierre de puño izquierdo (T1) y derecho (T2), con periodos de reposo (T0).
- Runs R03, R07, R11 (Tarea 1): Movimiento real de puño izquierdo (T1) y derecho (T2), con periodos de reposo (T0). NOTA: Los runs R05/R09/R13 corresponden a Tarea 3 (ambos pies o ambos punos) y quedan excluidos del analisis.

Los canales de interés corresponden a los electrodos sensoriomotores C3, C4 y Cz, ubicados sobre la corteza motora primaria izquierda, derecha y suplementaria, respectivamente. Estos electrodos son los más sensibles a los cambios en los ritmos μ (8–13 Hz) y β (13–30 Hz) asociados a la actividad motora [1].

3. Etapas Metodológicas

3.1 Preprocesamiento

Siguiendo la metodología validada en el Proyecto 1, el preprocesamiento incluirá:

1. Carga del archivo .edf con `mne.io.read_raw_edf()` y estandarización de nombres de canales con `mne.datasets.eegbci.standardize()`.
2. Selección de los canales C3, C4 y Cz mediante `raw.pick(['C3','C4','Cz'])`.
3. Aplicación de filtro Notch a 60 Hz para eliminar la interferencia de la red eléctrica de EE.UU.
4. Filtrado pasa-banda 8–30 Hz para aislar los ritmos sensoriomotores μ y β , eliminando derivas de baja frecuencia y ruido muscular de alta frecuencia [5].

5. Extracción de eventos mediante `mne.events_from_annotations()` y construcción de epochs con ventanas de $[-0.5, 4.0]$ s, con corrección de línea base en el intervalo $[-0.5, 0]$ s.
6. Rechazo de epochs con amplitud superior a $\pm 100 \mu\text{V}$ para minimizar artefactos de movimiento ocular y muscular.

3.2 Extracción de Características

Para cada epoch válido y cada canal de interés (C3, C4, Cz), se calcularán los siguientes índices:

a) Densidad Espectral de Potencia (PSD) — Proyecto 1

La potencia media en las bandas μ (8–13 Hz) y β (13–30 Hz) se calculará mediante el método de Welch con `scipy.signal.welch()`. Esta característica cuantifica la energía espectral en las bandas de interés y constituye la línea base de comparación con los índices adicionales [6].

b) Event-Related Desynchronization/Synchronization (ERD/ERS) — Proyecto 1

El porcentaje de cambio en la potencia de referencia permitirá identificar la desincronización (ERD) contralateral al miembro imaginado y la sincronización (ERS) ipsilateral, que son los biomarcadores principales de imaginación motora [5].

c) Coherencia Interhemisférica C3–C4 — Proyecto 1

Medida de la sincronización funcional entre los hemisferios izquierdo y derecho en las bandas μ y β , calculada con `scipy.signal.coherence()`. Permite analizar la conectividad funcional durante las tareas motoras [6].

d) Common Spatial Patterns (CSP) — Índice adicional

El CSP es una técnica de filtrado espacial que maximiza la varianza de una clase y la minimiza en la otra, generando componentes espaciales discriminativos para cada condición. Se implementará con `mne.decoding.CSP` de la librería MNE-Python. Las características se obtendrán como el logaritmo de la varianza de las señales proyectadas en los filtros CSP. Este método es especialmente efectivo para diferenciar imaginación de mano izquierda vs. derecha y reposo vs. tarea motora [2].

e) Entropía Espectral — Índice adicional

La entropía espectral cuantifica el grado de desorden de la señal EEG en el dominio de la frecuencia mediante la entropía de Shannon aplicada sobre la PSD normalizada. Valores bajos indican ritmos cerebrales más organizados (p. ej., en reposo con predominio del ritmo alfa), mientras que valores altos reflejan actividad más compleja durante imaginación motora. Se implementará con la librería `antropy` (función `spectral_entropy()`) o con `scipy.signal.welch()` combinado con cálculo manual de la entropía [7].

f) Parámetros de Hjorth — Índice adicional

Los tres parámetros de Hjorth describen la dinámica temporal de la señal EEG: (1) Actividad: varianza de la señal, proporcional a la potencia media; (2) Movilidad: raíz cuadrada de la relación entre la varianza de la primera derivada y la varianza de la señal, representando el contenido frecuencial promedio; (3) Complejidad: cociente entre la movilidad de la primera derivada y la movilidad de la señal, describiendo la variación en frecuencia a lo largo del tiempo. Se implementarán con la librería antropy (función `hjorth_params()`) o directamente con NumPy. Su bajo costo computacional los hace atractivos para aplicaciones en tiempo real [3].

3.3 Resumen de Condiciones e Índices Calculados

La siguiente tabla resume los índices que se calcularán para cada condición experimental en los tres canales de interés:

Condición	PSD μ/β	CSP	Ent. Espectral	Hjorth	ERD/ERS	Coherencia
Reposo (T0)	✓	Ref.	✓	✓	Ref.	✓
Imag. Derecha (T2)	✓	✓	✓	✓	✓	✓
Imag. Izquierda (T1)	✓	✓	✓	✓	✓	✓
Mov. Derecha (T2*)	✓	✓	✓	✓	✓	✓
Mov. Izquierda (T1*)	✓	✓	✓	✓	✓	✓

Nota: Ref. indica que la condición actúa como referencia basal en ese índice. (*) Condiciones adicionales para grupos de 3 integrantes.

3.4 Pipeline de Procesamiento Completo

Etapas	Descripción	Herramientas Python
1. Carga de datos	Leer archivos .edf del dataset Physionet BCI2000. Runs R04/R08/R12 (Tarea 2: imaginacion de mano izq./der.). Runs R03/R07/R11 (Tarea 1:	<code>mne.io.read_raw_edf()</code>

	movimiento real de mano izq./der.). Excluir R05/R06/R09/R10/R13/R14 (Tareas 3 y 4: pies y ambos punos)	
2. Selección de canales	Aislar electrodos sensoriomotores: C3, C4 y Cz	<code>raw.pick(['C3','C4','Cz'])</code>
3. Filtrado	Notch a 60 Hz (red eléctrica EE.UU.). Pasa-banda 8–30 Hz para ritmos μ y β	<code>mne.filter()</code> , <code>notch_filter()</code>
4. Eliminación de artefactos	Rechazo de epochs con amplitud $> \pm 100 \mu V$ para eliminar artefactos de movimiento y oculares	<code>mne.Epochs(reject=dict(eeg=100e-6))</code>
5. Segmentación (Epoching)	Extraer ventanas [-0.5, 4.0] s relativas a cada evento (T0, T1, T2). Baseline: [-0.5, 0] s	<code>mne.Epochs(tmin=-0.5, tmax=4.0, baseline=(-0.5,0))</code>
6. Extracción de características	Calcular para cada epoch y canal: PSD (potencia μ y β), CSP (log-varianza), Entropía espectral, Parámetros de Hjorth, ERD/ERS, Coherencia C3–C4	<code>mne.decoding.CSP</code> , <code>antropy</code> , <code>scipy.signal</code> , <code>numpy</code>
7. Almacenamiento en DataFrame	Construir tabla con columnas: Registro, Sujeto, Condición, Índice_Canal	<code>pandas.DataFrame</code>
8. Análisis estadístico	Estadística descriptiva (boxplots, violinplots). Prueba de normalidad (Shapiro-Wilk). ANOVA o Kruskal-Wallis según normalidad. Post-hoc Dunn o Tukey	<code>scipy.stats</code> , <code>pingouin</code> , <code>seaborn</code>
9. Clasificación	Selección de mejores características. Entrenamiento k-fold. Redes neuronales (MLP, CNN, LSTM). SVM y XGBoost. Matrices de confusión	<code>sklearn</code> , <code>tensorflow/keras</code> , <code>xgboost</code>

4. Estructura del DataFrame de Resultados

Todos los índices extraídos se almacenarán en un DataFrame de pandas con la siguiente estructura, tal como se especifica en el enunciado del proyecto:

- Registro: identificador único del archivo (p. ej., S001_R04).
- Sujeto: código del sujeto (S001 a S109).

- Condición/Tarea: etiqueta de la condición (Reposo, Imag_Derecha, Imag_Izquierda, Mov_Derecha, Mov_Izquierda).
- PSD_mu_C3, PSD_mu_C4, PSD_mu_Cz: potencia media en banda μ por canal.
- PSD_beta_C3, PSD_beta_C4, PSD_beta_Cz: potencia media en banda β por canal.
- CSP_1, CSP_2, ...: log-varianza de los componentes CSP más discriminativos.
- SpEnt_C3, SpEnt_C4, SpEnt_Cz: entropía espectral por canal.
- Hjorth_Act_C3, Hjorth_Mob_C3, Hjorth_Comp_C3 (y análogos para C4 y Cz).
- ERD_mu_C3, ERD_beta_C4, ...: porcentaje ERD/ERS por banda y canal.
- Coh_mu_C3C4, Coh_beta_C3C4: coherencia interhemisférica por banda.

5. Análisis Estadístico

5.1 Estadística Descriptiva

Para cada índice y condición se calcularán mediana, media, desviación estándar, mínimo y máximo. La representación visual se realizará mediante:

- Violin plots o boxplots: para comparar la distribución de cada índice entre las cinco condiciones. Preferibles a diagramas de barras por mostrar la distribución completa [8].
- Heatmaps de correlación: para identificar redundancia entre índices y guiar la selección de características para clasificación.
- Topographic plots (usando MNE): para visualizar la distribución espacial de los índices en los electrodos motores.

5.2 Pruebas de Hipótesis

Se plantearán hipótesis para cada índice calculado. El esquema general es:

H₀: No existen diferencias estadísticamente significativas en el índice X entre las condiciones evaluadas (Reposo, Imag_Derecha, Imag_Izquierda, Mov_Derecha, Mov_Izquierda).

H₁: Existe al menos una diferencia estadísticamente significativa en el índice X entre las condiciones evaluadas.

La selección del tipo de prueba se realizará según el siguiente criterio:

7. Prueba de normalidad (Shapiro-Wilk) para cada grupo de datos. Si $p > 0.05$ se asume normalidad.
8. Si todos los grupos siguen distribución normal: ANOVA de una vía seguido de prueba post-hoc de Tukey HSD para comparaciones múltiples.
9. Si algún grupo no sigue distribución normal: prueba no paramétrica de Kruskal-Wallis, seguida de prueba post-hoc de Dunn con corrección de Bonferroni.
10. Nivel de significancia: $\alpha = 0.05$. Se reportarán los valores p y el tamaño del efecto (η^2 o ϵ^2).

6. Normalización de Características

Antes de entrenar cualquier modelo de clasificación, es indispensable normalizar las características extraídas. Esto se debe a que los índices calculados operan en escalas numéricas muy diferentes entre sí. Por ejemplo, la potencia espectral (PSD) toma valores del orden de 10^{-11} V^2 , la entropía espectral varía entre 0 y 1, y los parámetros de Hjorth son adimensionales con rangos distintos según el canal. Si se entrena un clasificador con estas características sin normalizar, el modelo asignará mayor peso implícito a las variables con valores numéricamente más grandes, no porque sean más discriminativas biológicamente, sino simplemente por su escala. Esto distorsiona el aprendizaje y reduce el desempeño del clasificador [9].

6.1 Métodos de Normalización

a) Estandarización Z-score (StandardScaler)

Es el método más utilizado en clasificación de señales EEG. Transforma cada característica para que tenga media cero y desviación estándar uno, aplicando la fórmula:

$$x_{norm} = (x - \mu) / \sigma$$

donde μ es la media y σ la desviación estándar de cada característica calculadas exclusivamente sobre el conjunto de entrenamiento. Este método es el más adecuado cuando las características no siguen una distribución uniforme conocida y es el recomendado para SVM y redes neuronales, ya que estos modelos son especialmente sensibles a la escala de las variables de entrada [9]. Se implementa con `sklearn.preprocessing.StandardScaler`.

b) Normalización Min-Max (MinMaxScaler)

Escala cada característica al rango [0, 1] mediante:

$$x_{norm} = (x - x_{min}) / (x_{max} - x_{min})$$

Es útil cuando se requiere que todas las variables estén en el mismo rango acotado, como en capas de activación sigmoid o para visualización comparativa. Sin embargo, es más sensible a valores atípicos (outliers) que el Z-score, lo cual puede ser problemático dado que las señales EEG presentan artefactos residuales. Se implementa con `sklearn.preprocessing.MinMaxScaler`.

6.2 Codificación de Etiquetas (Label Encoding)

Los modelos de clasificación no pueden procesar directamente etiquetas de texto como Reposo, Imag_Derecha o Mov_Izquierda. Es necesario convertirlas a valores numéricos enteros

mediante un proceso denominado Label Encoding, que asigna un número único a cada clase. Para este proyecto la codificación propuesta es:

- 0 → Reposo (T0)
- 1 → Imaginación mano izquierda (T1 - Tarea 2)
- 2 → Imaginación mano derecha (T2 - Tarea 2)
- 3 → Movimiento real mano izquierda (T1 - Tarea 1)
- 4 → Movimiento real mano derecha (T2 - Tarea 1)

Esta codificación se implementa con `sklearn.preprocessing.LabelEncoder` o simplemente mapeando el `DataFrame` con un diccionario de Python. Es importante distinguir este proceso de la normalización: el Label Encoding se aplica sobre la columna de etiquetas (variable objetivo *y*), mientras que la normalización se aplica sobre las columnas de características (variables de entrada *X*). Una confusión frecuente es aplicar `StandardScaler` también sobre las etiquetas, lo cual es un error ya que los números asignados a las clases no tienen relación de magnitud entre sí.

6.3 Consideración Crítica: Evitar Data Leakage

Es fundamental que los parámetros de normalización (media, desviación estándar, mínimo y máximo) se calculen ÚNICAMENTE sobre los datos de entrenamiento de cada fold en la validación cruzada *k-fold*, y luego se apliquen sobre los datos de validación y prueba. Calcular estos parámetros sobre el conjunto completo antes de dividir los datos constituye un error metodológico grave llamado *data leakage*, que produce estimaciones de desempeño artificialmente optimistas y no representativas del rendimiento real del clasificador sobre datos nuevos [9]. En la implementación con `scikit-learn`, esto se garantiza utilizando `Pipeline`, que encadena el escalador y el clasificador de forma que la normalización se recalcula en cada fold automáticamente.

6.3 Aplicación por Índice

En este proyecto se aplicará `StandardScaler` como método principal de normalización, dado que los modelos implementados (SVM, redes neuronales MLP, CNN, LSTM) son sensibles a la escala. `XGBoost` es el único clasificador que, por su naturaleza basada en árboles de decisión, no requiere estrictamente normalización, aunque normalizarlos no afecta negativamente su desempeño. La normalización se aplicará sobre el `DataFrame` completo de características justo antes del paso de clasificación, dentro del esquema de validación cruzada *k-fold*.

7. Estrategia de Clasificación

7.1 Selección de Características

El enunciado del proyecto establece explícitamente que la clasificación debe realizarse con los índices que muestren mayor diferencia entre condiciones. Por tanto, no se usarán todas las

características extraídas, sino únicamente aquellas que superen los siguientes criterios de selección, aplicados sobre los resultados del análisis estadístico de la Sección 5:

Criterio 1: Significancia estadística

Se conservarán únicamente las características cuya prueba de Kruskal-Wallis (o ANOVA, según normalidad) arroje un p-valor menor a 0.05 entre las cinco condiciones experimentales. Las características con $p \geq 0.05$ se descartan por no aportar poder discriminativo estadísticamente verificable.

Criterio 2: Tamaño del efecto

Entre las características que superen el Criterio 1, se priorizarán aquellas con mayor tamaño del efecto. Para Kruskal-Wallis se usará el coeficiente eta cuadrado (η^2), y para ANOVA el eta cuadrado parcial (η^2_p). Se considera un efecto grande cuando $\eta^2 > 0.14$. Esto garantiza que la diferencia no solo sea estadísticamente significativa sino también clínicamente o fisiológicamente relevante.

Criterio 3: Selección univariada con ANOVA F-score

Como paso complementario, se aplicará SelectKBest con `f_classif` de scikit-learn para rankear todas las características según su F-score univariado. Se seleccionarán las k características con mayor puntuación, donde k se determinará comparando el desempeño del clasificador para $k = 5, 10, 15$ y el total de características, eligiendo el k que maximice el F1-score en validación cruzada. Esto se implementa con `sklearn.feature_selection.SelectKBest`.

El resultado de este proceso será una lista ordenada de características seleccionadas, que se documentará explícitamente indicando para cada una el índice, el canal, el p-valor, el tamaño del efecto y el F-score, justificando así de forma transparente por qué entraron al clasificador y por qué se descartaron las demás.

7.2 Estrategia de Entrenamiento: K-Fold Cross-Validation

Para garantizar una evaluación robusta del rendimiento, se implementará la validación cruzada k -fold con $k = 5$ o $k = 10$. En cada iteración, $k-1$ folds se usarán para entrenamiento y 1 fold para validación. Las métricas reportadas serán la media y desviación estándar de la exactitud (accuracy), F1-score, sensibilidad y especificidad a través de los folds. Se considerará también la validación Leave-One-Subject-Out (LOSO) para evaluar la generalización entre sujetos.

7.3 Arquitecturas de Clasificación

Se implementarán y compararán al menos cinco clasificadores:

11. Perceptrón Multicapa (MLP): red neuronal completamente conectada con capas ocultas de activación ReLU y capa de salida softmax para clasificación multiclase. Implementación con `sklearn.neural_network.MLPClassifier` o TensorFlow/Keras.

12. Red Neuronal Convolutacional 1D (CNN): extracción automática de características temporales mediante capas convolucionales aplicadas sobre los epochs de EEG. Arquitectura: Conv1D → BatchNorm → MaxPooling → Dense → Softmax. Implementación con TensorFlow/Keras.
13. Red Neuronal Recurrente (LSTM): captura de dependencias temporales en la señal EEG mediante celdas de memoria a largo plazo. Implementación con TensorFlow/Keras.
14. Máquina de Soporte Vectorial (SVM): clasificador con kernel RBF (Radial Basis Function) que maximiza el margen de separación entre clases en el espacio de características. Implementación con `sklearn.svm.SVC`. Se optimizarán los hiperparámetros C y γ mediante búsqueda en rejilla (`GridSearchCV`).
15. Extreme Gradient Boosting (XGBoost): método de ensamble basado en árboles de decisión con gradiente potenciado, con regularización L1/L2 para evitar sobreajuste. Implementación con la librería `xgboost`.

7.4 Evaluación del Desempeño

Los clasificadores se compararán mediante las siguientes métricas y herramientas:

Métricas cuantitativas

- Accuracy (exactitud global): proporción de muestras clasificadas correctamente sobre el total. Útil como métrica general pero puede ser engañosa si las clases están desbalanceadas.
- Precision por clase: de todas las muestras que el modelo predijo como clase X, cuántas realmente eran clase X. Relevante para evitar falsas alarmas en aplicaciones BCI.
- Recall (sensibilidad) por clase: de todas las muestras que realmente son clase X, cuántas identificó correctamente el modelo. Crítico en BCI porque un recall bajo en una clase significa que el sistema falla en detectar esa intención de movimiento.
- F1-score por clase: media armónica entre precision y recall. Se reportará como promedio macro (igual peso a todas las clases) y ponderado (peso proporcional al número de muestras por clase).

Matrices de confusión

Se generará una matriz de confusión normalizada para cada clasificador, donde cada celda muestra la proporción de muestras de la clase real (fila) que fueron predichas como cada clase (columna). A partir de estas matrices se discutirán los siguientes patrones esperados fisiológicamente:

- Confusión entre Imaginación mano derecha y Movimiento real mano derecha: ambas condiciones activan la misma región cortical (C3 contralateral) y generan patrones ERD similares, por lo que se espera que sean el par de clases más difícil de separar.
- Confusión entre Imaginación mano izquierda y Movimiento real mano izquierda: por la misma razón anterior pero en el hemisferio derecho (C4).
- Separación correcta entre condiciones de mano derecha e izquierda: se espera buena discriminación gracias al patrón de lateralización C3-C4, que es la característica más discriminativa según el análisis del Proyecto 1.

- Reposo bien identificado: el estado de reposo presenta la mayor potencia en bandas μ y β y ausencia de ERD, lo que lo hace la clase más fácil de separar del resto.

Curvas ROC y AUC

Para cada clasificador se calcularán curvas ROC usando una estrategia One-vs-Rest (OvR), generando una curva por cada una de las 5 clases. El área bajo la curva (AUC) servirá como métrica resumen del poder discriminativo por clase, donde $AUC = 1.0$ indica clasificación perfecta y $AUC = 0.5$ indica desempeño equivalente al azar. Se implementa con `sklearn.metrics.roc_curve` y `roc_auc_score`.

Comparación entre clasificadores

Se construirá una tabla resumen con las métricas de todos los clasificadores (MLP, CNN, LSTM, SVM, XGBoost) para facilitar la comparación directa. La selección del mejor modelo se basará principalmente en el F1-score macro promedio sobre los folds de validación cruzada, ya que esta métrica balancea precision y recall sin favorecer clases mayoritarias.

9. Referencias

- [1] H. Ramoser, J. Muller-Gerking y G. Pfurtscheller, Optimal spatial filtering of single trial EEG during imagined hand movement, *IEEE Trans. Rehab. Eng.*, vol. 8, no. 4, pp. 441-446, 2000.
- [2] C. Wu et al., The Application of Entropy in Motor Imagery Paradigms of Brain-Computer Interfaces, *Brain Sciences*, vol. 15, no. 2, p. 168, 2025. doi: 10.3390/brainsci15020168.
- [3] W. H. Alawee, A. Basem y L. A. Al-Haddad, Leveraging Hjorth features for EEG signal analysis, *J. Electr. Bioimpedance*, vol. 14, no. 1, pp. 66-72, 2023. doi: 10.2478/joeb-2023-0009.
- [4] G. Schalk et al., BCI2000: A general-purpose brain-computer interface system, *IEEE Trans. Biomed. Eng.*, vol. 51, no. 6, pp. 1034-1043, 2004.
- [5] G. Pfurtscheller y F. H. Lopes da Silva, Event-related EEG/MEG synchronization and desynchronization: basic principles, *Clin. Neurophysiol.*, vol. 110, pp. 1842-1857, 1999.
- [6] P. D. Welch, The use of fast Fourier transform for the estimation of power spectra, *IEEE Trans. Audio Electroacoust.*, vol. 15, no. 2, pp. 70-73, 1967.
- [7] R. Vallat, Antropy: Entropy and complexity of EEG signals in Python, PyPI. <https://pypi.org/project/antropy/>
- [9] A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 2nd ed. Sebastopol, CA: O'Reilly Media, 2019. ISBN: 978-1-492-03264-9.
- [8] Dataviz Project, From data to viz - find the right chart type, 2024. <https://www.data-to-viz.com/>